AFFORDABLE HOUSING DOCUMENT PLATFORM & MERGE-FIELD MIGRATION

Engineering Case Study

Domain	Affordable Housing / Enterprise Document Generation
System	Customer-facing document generation and merge-field resolution
Status	Historical production system and completed schema migration
Primary concerns	Correctness, backward compatibility, auditability, operational continuity
Document purpose	Technical case study based on direct module ownership and production support

Scope note

This document records the architecture, migration responsibilities, production risks, and operational lessons described by the author. The precise proprietary schema names, customer data, and internal identifiers are intentionally omitted. Another engineering team owned the broader schema redesign; the author owned the Affordable Housing document-module integration, mapping validation, release support, and production troubleshooting.

Contents

1. Executive Summary

2. Business and Domain Context

3. Problem Statement and System Goals

4. Ownership and System Boundaries

5. Functional and Non-Functional Requirements

6. Existing Document Generation Architecture

7. Merge-Field Model and Dependency Chain

8. Migration Drivers and Target Design

9. Backward Compatibility Strategy

10. Data Mapping and Transformation

11. Migration Workflow and Implementation

12. Validation and Test Strategy

13. Release and Cutover Strategy

14. Production Incidents and Triage

15. Reliability and Failure Modes

16. Observability and Operational Controls

17. Security, Privacy, and Auditability

18. Design Decisions and Trade-offs

19. Lessons Learned

20. Modernization Direction

Appendix A. Migration Checklist

Appendix B. Glossary

1. Executive Summary

The Affordable Housing document platform produced customer-facing forms and notices by combining document templates, merge-field definitions, business data, and a field-resolution layer. The platform supported a large and mature catalog of templates accumulated over years of product development. Hundreds of documents could be generated in normal daily operation, so the correctness of every mapping was directly visible to customers.

A separate engineering team redesigned the merge-field database model. The redesign improved the underlying representation, but it created a high-risk integration problem for the Affordable Housing module. Existing templates referenced legacy merge fields, new templates depended on the redesigned definitions, and a mapping error could affect both previously established workflows and newly generated documents.

The author owned the Affordable Housing portion of this migration. That ownership included understanding the module-specific document types, analyzing the legacy-to-target mapping, coordinating with the team responsible for the new schema, validating generated output, supporting QA, participating in release execution, and resolving urgent production defects. The work required both database-level reasoning and detailed knowledge of the document-generation domain.

The key engineering challenge was not simply copying records from an old table to a new table. It was preserving semantic compatibility. A merge field represents a contract between the template, the field catalog, the resolver, the underlying business data, and the final rendered document. Migration therefore required validation at every layer: metadata, mapping, data resolution, template behavior, and rendered output.

A production period in which six urgent defects arrived in one day demonstrated the operational sensitivity of the system. Those incidents were treated as symptoms of mapping, compatibility, or data-resolution defects rather than as unrelated document bugs. The experience reinforced the need for canonical mappings, automated comparison, representative test fixtures, phased rollout, and a durable audit trail.

Document Generation Dependency Flow

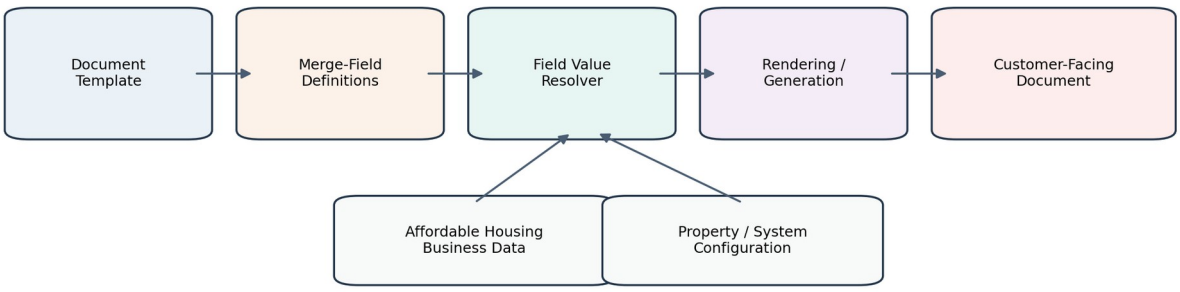


Figure 1. The document platform depends on a chain of contracts; a defect in any link can produce incorrect customer output.

2. Business and Domain Context

2.1 Role of documents in Affordable Housing

Affordable Housing software supports regulated workflows in which customer communication and compliance evidence are often represented as generated documents. Documents may reflect household information, certification data, property configuration, financial values, effective dates, eligibility-related details, or program-specific language. They are not cosmetic output; they are part of the operational record used by property teams and residents.

2.2 Why merge fields exist

Templates provide stable wording and layout, while merge fields inject context-specific values at generation time. This separation allows one template to serve many properties and households. A field can represent a simple value, such as a property name, or a derived value that depends on business rules, effective dates, or related entities.

2.3 Customer impact

- An unresolved field can leave an empty placeholder or expose an internal token in a final document.
- An incorrect mapping can display the wrong value while still producing a technically valid file.
- A formatting difference can alter dates, currency, percentages, names, or regulatory text.
- A broken template-field relationship can block generation completely.
- A migration defect can affect both old and new documents if compatibility behavior is incomplete.
- Regeneration can become necessary when an incorrect document has already been produced or distributed.

Operational consequence
The cost of a defect is not limited to fixing code. It can include identifying affected documents, determining whether they were delivered, correcting the underlying mapping, regenerating output, communicating with support or customers, and proving that the corrected system is complete.

3. Problem Statement and System Goals

The platform needed to adopt a redesigned merge-field data model without breaking mature Affordable Housing document workflows. The migration had to preserve the meaning of existing fields, support new definitions, avoid silent value substitution, and provide a controlled path for testing and production cutover.

3.1 Primary goals

- Preserve the semantic meaning of legacy merge fields after migration.

- Allow existing templates to continue generating correct output during the transition.
- Enable new templates and new field definitions to use the redesigned model.
- Detect missing, duplicate, ambiguous, or incorrectly transformed mappings before production.
- Validate actual rendered documents, not only database row counts.
- Provide a reversible and observable release process.
- Reduce the time required to diagnose production document defects.

3.2 Non-goals

- Redesigning every document template as part of the database migration.
- Changing Affordable Housing business rules unrelated to document resolution.
- Replacing the rendering engine solely because the metadata model changed.
- Claiming ownership of the enterprise-wide schema redesign; ownership was limited to the Affordable Housing integration and operational outcome.

4. Ownership and System Boundaries

The migration crossed team boundaries. One team owned the redesigned database model and the common merge-field infrastructure. The author owned the Affordable Housing document module and therefore owned the correctness of its integration with the new model.

Area	Ownership and responsibility
Common schema redesign	Owned by another engineering team. Included the redesigned merge-field representation and shared infrastructure.
Affordable Housing field inventory	Owned by the author. Required identifying module-specific fields, templates, and resolver behavior.
Legacy-to-target mapping	Collaborative. The common model was provided by the platform team; module-specific semantic validation belonged to the author.
Template validation	Owned within the Affordable Housing scope, with QA support and representative customer scenarios.
Production release support	Shared across engineering, QA, database, and release stakeholders. The author investigated module-specific failures.
Customer issue resolution	Owned operationally for the affected module, including root-cause analysis and corrective validation.

This boundary is important because senior ownership does not require claiming every layer. It requires understanding dependencies, making the integration safe, and taking responsibility for the outcome within the owned domain.

5. Functional and Non-Functional Requirements

5.1 Functional requirements

- Maintain a catalog of merge-field definitions and their supported value sources.
- Resolve fields against Affordable Housing business data at generation time.
- Support legacy templates during the migration window.
- Support new templates against the redesigned schema.
- Map each legacy field to a canonical target or explicitly mark it as retired.
- Preserve data type, formatting, and null-handling semantics.
- Generate a final document with no unresolved placeholders.
- Record enough information to trace a generated value back to its definition and source.

5.2 Non-functional requirements

- Correctness: the generated value must represent the same business meaning before and after migration.
- Backward compatibility: established templates must continue working until intentionally retired.
- Auditability: mapping decisions and production changes must be reviewable.
- Availability: routine document generation should remain available during rollout.
- Performance: the new lookup path should not introduce unacceptable generation latency.
- Operability: support and engineering teams need actionable evidence when a document fails.
- Security: document values must respect property, customer, and user access boundaries.

6. Existing Document Generation Architecture

The document platform can be understood as a layered pipeline. A user or workflow selects a template. The template contains merge-field references. A field catalog identifies each field and its resolver contract. The resolver retrieves or derives the appropriate business value. The rendering component substitutes values into the template and produces the final document.

6.1 Core components

Component	Responsibility
Template repository	Stores the document layout, static wording, and field placeholders.
Merge-field catalog	Defines supported fields, identifiers, names, value types, and resolver relationships.
Field resolver	Retrieves direct values or executes business-aware derivation logic.

Component	Responsibility
Business data layer	Supplies household, property, certification, financial, and configuration context.
Renderer	Combines the template and resolved values into a customer-facing file.
Document history	Tracks generated output and supports operational investigation or regeneration.

6.2 Coupling risks

The architecture contains several forms of coupling. Templates may refer to fields by identifier or token. Resolvers may assume a field type or metadata shape. Business logic may depend on the selected document context. Formatting rules may exist in either the resolver or the rendering layer. A database redesign can therefore break behavior even when all migrated rows appear present.

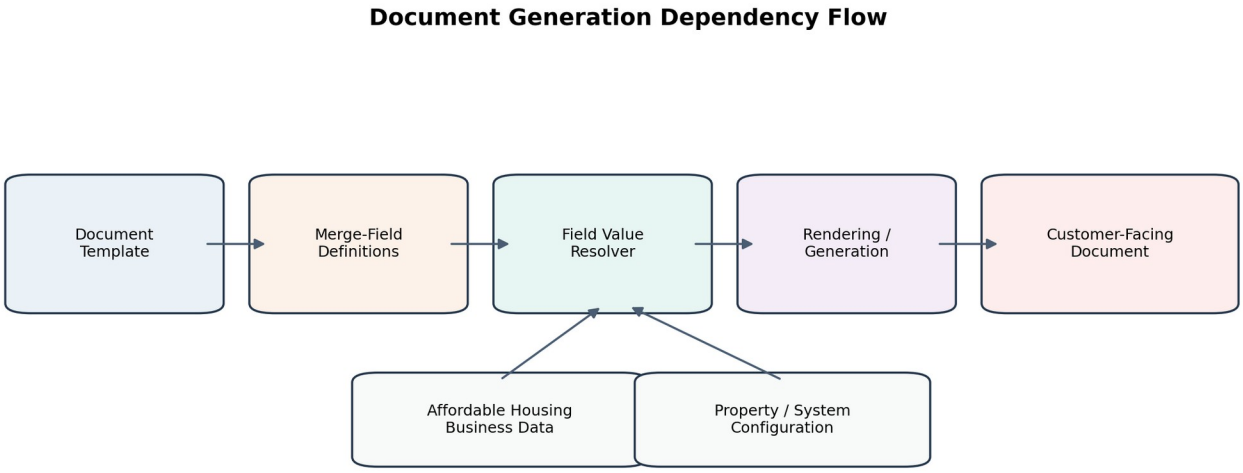


Figure 2. End-to-end architecture from template selection through generated customer output.

7. Merge-Field Model and Dependency Chain

7.1 Merge fields as contracts

A merge field is best treated as a versioned contract rather than a text placeholder. Its contract includes a stable identity, a human-readable name, the source of its value, expected data type, formatting rules, null behavior, supported document contexts, and ownership.

7.2 Field categories

- Direct fields: values read from a single authoritative record.
- Relational fields: values resolved through related property, household, or certification records.
- Derived fields: values calculated from multiple inputs or business rules.
- Conditional fields: values or text selected based on program, property, or workflow state.

- Collection fields: repeated rows, household members, income sources, or other variable-length groups.
- Formatted fields: dates, currency, percentages, addresses, and names with display-specific behavior.

7.3 Sources of semantic drift

- A legacy field name and a new field name look similar but represent different effective dates.
- A field changes from a direct source to a derived source without preserving edge-case behavior.
- Null and empty-string behavior changes after migration.
- Formatting moves between resolver and template layers and is applied twice or not at all.
- A common field exists globally, but the Affordable Housing module requires a program-specific variant.
- Multiple legacy fields collapse into one target field even though their historical usage differs.

8. Migration Drivers and Target Design

The redesigned schema was intended to create a more maintainable and consistent foundation for merge-field definitions. The migration opportunity was to replace accumulated legacy relationships with a clearer model while preserving production behavior.

8.1 Target design characteristics

- Canonical field definitions with clear ownership and stable identifiers.
- Explicit relationships between fields, resolvers, modules, and document contexts.
- Separation of field metadata from customer-specific values.
- Ability to version, deprecate, and audit definitions.
- Consistent query and resolution behavior across modules.
- Fewer hidden dependencies on legacy table structure.

8.2 Migration principle

Principle
Migrate meaning, not rows. Record-count parity is necessary but insufficient; the new system must produce equivalent business output for representative scenarios.

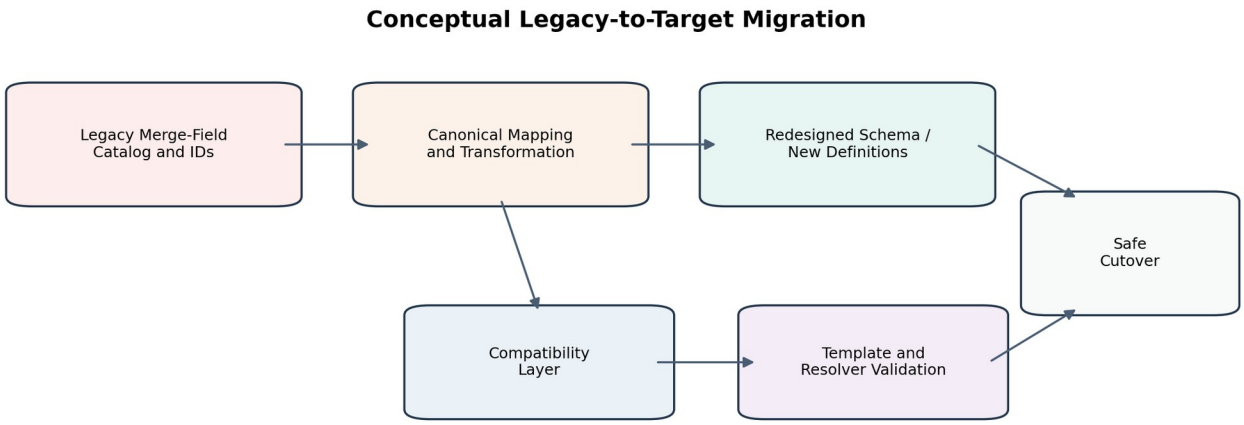


Figure 3. The migration uses a canonical mapping and compatibility layer before safe cutover.

9. Backward Compatibility Strategy

Backward compatibility was necessary because existing templates could not all be rewritten and re-approved at the same time. The migration therefore needed an intermediate contract that allowed legacy references to resolve through the new model.

9.1 Compatibility options

Option	Trade-off
Direct in-place replacement	Simple final state, but high cutover risk because all templates and references must change together.
Dual-read compatibility	Allows legacy and new definitions to coexist, but increases temporary complexity and requires deterministic precedence.
Alias or mapping layer	Preserves legacy tokens while routing them to canonical definitions. Requires strong mapping governance.
Template-by-template migration	Reduces blast radius but extends the migration window and requires tracking mixed states.
Full template regeneration	Provides a clean reset but is operationally expensive and risky for mature customer-specific templates.

9.2 Chosen conceptual approach

The practical approach was to establish explicit legacy-to-target mappings, validate compatibility within the Affordable Housing module, and use controlled rollout rather than assuming that the redesigned schema alone guaranteed equivalent behavior. The exact implementation details were part of the proprietary system, but the engineering principle was consistent: legacy references required an intentional path into the new model.

9.3 Compatibility exit criteria

- All active templates have known field dependencies.
- Every legacy field is mapped, intentionally retired, or isolated for manual review.
- Rendered output is equivalent for representative scenarios.
- Production telemetry no longer shows legacy-only resolution paths.
- Operational teams have a documented rollback and reconciliation process.

10. Data Mapping and Transformation

10.1 Inventory

The first technical task was creating an inventory of module-specific templates and field definitions. The inventory needed to identify not only field records but also where they were referenced and how their values were resolved.

10.2 Canonical mapping record

Attribute	Purpose
Legacy identifier or token	Provides traceability from existing templates and historical behavior.
Target canonical field	Identifies the new definition that preserves the business meaning.
Resolver or source	Documents how the value is obtained or derived.
Data type and format	Protects date, currency, numeric, boolean, and display semantics.

Attribute	Purpose
Supported context	Restricts the field to valid modules, templates, programs, or document types.
Migration status	Tracks mapped, validated, deprecated, exception, and blocked states.
Evidence	Links sample documents, tests, or review notes supporting the mapping decision.

10.3 Transformation controls

- Uniqueness checks for identifiers, aliases, and canonical mappings.
- Referential-integrity checks between definitions and resolver metadata.
- Explicit treatment of fields with no valid target.
- Detection of multiple legacy fields mapped to one target without approved equivalence.
- Validation of data type and formatting compatibility.
- Reconciliation of active template references before cutover.

The transformation was therefore both a data migration and a semantic review. Fields that could not be proven equivalent required an exception path rather than an optimistic mapping.

11. Migration Workflow and Implementation

1. Create the Affordable Housing inventory of active templates, fields, and resolver dependencies.
2. Obtain the redesigned schema contract and common migration approach from the owning platform team.
3. Build or review legacy-to-target mappings for module-specific fields.
4. Load or transform definitions into the target model in a controlled environment.
5. Run static reconciliation for counts, uniqueness, references, and unmapped fields.
6. Generate representative documents using both legacy and target resolution paths where possible.
7. Compare values, formatting, optional sections, and repeated data.
8. Correct mapping or resolver defects and repeat validation.
9. Coordinate QA coverage across document types and business scenarios.
10. Execute a phased production release with smoke tests and rapid rollback capability.
11. Monitor support signals and reconcile any production defects to the migration inventory.

The workflow was iterative. A failed rendered document was not treated as merely a template problem; it could indicate an incorrect field definition, an incomplete mapping, a resolver change, a missing data relationship, or a format mismatch.

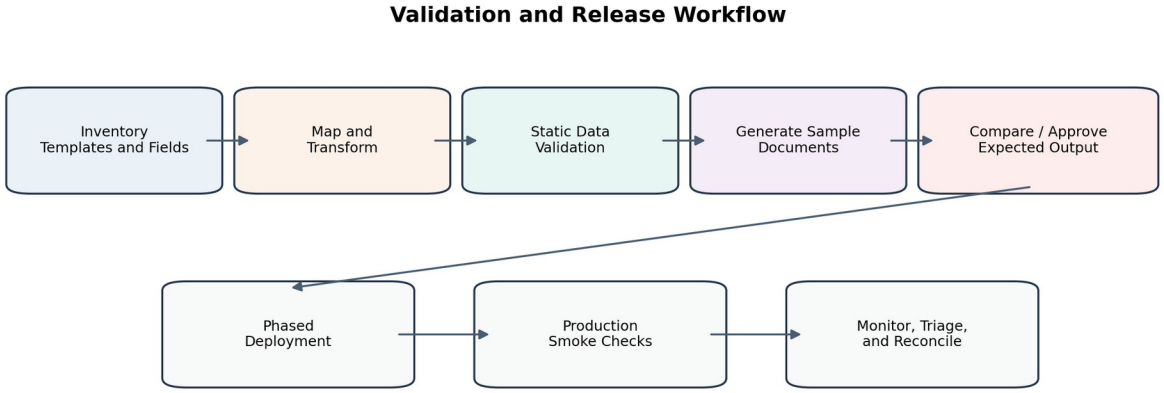


Figure 4. Migration workflow from inventory through production reconciliation.

12. Validation and Test Strategy

12.1 Layered validation

Layer	Validation focus
Schema validation	Required columns, constraints, relationships, and accepted data types.
Mapping validation	Completeness, uniqueness, approved exceptions, and semantic equivalence.
Resolver validation	Correct source record, business context, derived logic, and null handling.
Template validation	Valid field references, supported contexts, and no unresolved placeholders.
Rendered-output validation	Visible values, formatting, repeated sections, pagination, and conditional content.
Operational validation	Generation success rate, latency, error visibility, history records, and regeneration behavior.

12.2 Representative fixtures

A strong test set must represent the diversity of real document behavior. A small happy-path sample is insufficient because many field defects appear only with missing optional data, multiple household members, date boundaries, property-specific settings, or older templates.

- Minimal and fully populated household scenarios.
- Single and multiple related records for repeated sections.
- Null, empty, zero, and not-applicable values.
- Date and effective-period boundaries.
- Property-specific configuration differences.
- Legacy templates with long production history.
- New templates using only canonical definitions.
- Known historical defects converted into regression cases.

12.3 Document comparison

Database comparison verifies that definitions exist; document comparison verifies that customers see the correct result. Validation should compare field values and visible rendering while allowing expected non-semantic differences such as metadata or renderer-generated identifiers.

Testing lesson
The final document is the business artifact. A migration cannot be considered safe until representative documents have been generated and reviewed end to end.

13. Release and Cutover Strategy

13.1 Pre-release controls

- Freeze or track changes to affected definitions during the migration window.
- Reconcile active template references against the approved mapping set.
- Prepare deployment scripts and rollback scripts as a reviewed pair.
- Capture baseline generation results for high-value document scenarios.
- Define smoke tests that can be completed immediately after deployment.
- Assign clear owners for database, application, QA, and module-level decisions.

13.2 Phased rollout

A phased rollout limits the number of affected templates or contexts at each step. Where the platform permits, a compatibility path or feature control can preserve legacy behavior while the target path is validated. Phasing is especially valuable when the document catalog is large and different modules have different readiness levels.

13.3 Rollback

Rollback must restore both data and resolution behavior. Reverting application code without reverting incompatible definitions can leave the platform in a mixed state. The rollback plan therefore needs an explicit compatibility matrix describing which application version supports which schema and mapping version.

13.4 Post-release reconciliation

- Generate predefined smoke-test documents.
- Confirm no unresolved or unknown field references.
- Compare key field values with baseline expectations.
- Review generation errors by template and field.
- Monitor support-ticket volume and document-regeneration requests.
- Update the migration inventory with any discovered exception.

14. Production Incidents and Triage

The migration generated urgent operational work because document defects were customer-visible and could affect both existing and newly created output. On one day, six urgent bugs required investigation. The important engineering response was to avoid treating each defect as isolated and instead look for shared failure patterns across mapping, resolution, and compatibility behavior.

14.1 Triage sequence

1. Identify the exact template, document type, property context, and generation time.
2. Capture the unresolved, missing, or incorrect field and the expected business value.
3. Trace the template reference to the legacy and target field definitions.
4. Validate the mapping and determine which resolution path executed.
5. Inspect the authoritative business data and relevant configuration.
6. Reproduce the issue with the same context in a controlled environment.
7. Determine whether the defect affects one template, one field, one module, or the common platform.
8. Correct the root cause and identify documents that may require regeneration.
9. Add the scenario to regression coverage and update migration documentation.

14.2 Evidence required

- Generated document identifier and template version.
- Field token or identifier before and after migration.
- Resolver selection and source-data record.
- Expected versus actual value and formatting.
- Application and schema versions active at generation time.
- Whether the issue reproduces through legacy, target, or both paths.
- Count or query identifying potentially affected documents.

14.3 Regeneration considerations

Correcting the mapping does not automatically correct documents already generated. The response process must determine which outputs are affected, whether they were delivered, whether regeneration is legally or operationally appropriate, and how the corrected document will be recorded without destroying the audit history.

15. Reliability and Failure Modes

Failure mode	Control or response
Missing field mapping	Block cutover for active references; require an explicit exception for retired fields.
Incorrect semantic mapping	Use representative rendered comparisons and business-owner review.
Duplicate or ambiguous alias	Enforce uniqueness and deterministic precedence.
Resolver returns no value	Differentiate valid null from missing data or lookup failure.
Formatting regression	Test dates, money, percentages, names, and multiline values end to end.
Mixed application/schema versions	Maintain a compatibility matrix and coordinated deployment order.
Partial migration	Track migration state and reconcile every active definition.
Common-platform defect	Escalate across team boundaries with reproducible evidence and blast-radius analysis.
Incorrect documents already generated	Identify affected output, preserve audit history, and use controlled regeneration.
Support overload after cutover	Prepare diagnostic queries, runbooks, and rapid ownership routing.

15.1 Silent failures are the highest risk

A hard generation failure is disruptive but visible. A plausible-looking document with the wrong value is more dangerous because it may pass through normal workflows. Validation therefore needs to test correctness, not merely completion.

15.2 Idempotent migration operations

Migration scripts and reconciliation steps should be safe to rerun. Idempotency reduces recovery risk when a deployment is interrupted or a corrected mapping set must be applied again. Repeated execution should not create duplicate definitions or change already validated records unexpectedly.

16. Observability and Operational Controls

Document systems require observability at the field, template, and generation levels. A generic application error is not enough for support teams to diagnose a failure quickly.

16.1 Recommended metrics and logs

- Document generation success and failure counts by template and module.
- Unknown or unresolved merge-field references.
- Null-value frequency for fields that historically resolve values.
- Resolver latency and error rate.

- Legacy versus target resolution-path usage during migration.
- Top failing templates and fields after deployment.
- Regeneration volume and reason codes.
- Count of active legacy mappings remaining.

16.2 Correlation identifiers

Every generation should have a correlation identifier connecting the user action or workflow, template version, field-resolution events, renderer execution, and stored document record. This allows an engineer to reconstruct what the system knew at the moment the document was produced.

16.3 Operational runbook

- How to find the active template and its field references.
- How to inspect the legacy-to-target mapping.
- How to reproduce a generation using safe test data.
- How to determine whether a field is direct, derived, conditional, or repeated.
- How to identify affected documents after a mapping defect.
- When to regenerate, rollback, or escalate to the common platform team.

17. Security, Privacy, and Auditability

Affordable Housing documents can contain sensitive customer, resident, property, and financial information. The migration cannot weaken the authorization boundaries already enforced by the platform.

- Field resolution must use the same tenant, property, and user authorization context as the document request.
- Migration and diagnostic tooling should avoid exposing full sensitive values unless required for authorized support.
- Generated documents and regeneration events should retain an audit history.
- Definition changes should record who changed the mapping, when, and why.
- Production troubleshooting exports should be minimized, secured, and removed after use.
- Test fixtures should use synthetic or appropriately masked data.

Auditability is also an engineering requirement. When a document is questioned, the team should be able to determine the template version, field-definition version, input context, and generation timestamp without reconstructing the system from incomplete logs.

18. Design Decisions and Trade-offs

Decision	Reasoning and accepted trade-off
Preserve compatibility during migration	Reduced immediate customer risk, but temporarily increased system complexity.
Use explicit canonical mappings	Made decisions reviewable and auditable, but required

Decision	Reasoning and accepted trade-off
	significant domain analysis.
Validate rendered output	Provided the strongest correctness signal, but increased test preparation and review effort.
Separate platform and module ownership	Allowed expertise at each layer, but required disciplined cross-team communication.
Use phased rollout	Limited blast radius, but extended the period in which mixed states had to be operated.
Treat urgent bugs as migration signals	Improved systemic root-cause analysis, but required pausing local fixes long enough to search for common causes.
Retain deterministic document logic	Protected regulated output from probabilistic behavior and preserved explainability.

18.1 What was deliberately avoided

- Assuming row-count parity proved functional equivalence.
- Rewriting every template in one release.
- Changing unrelated business logic during a high-risk schema migration.
- Allowing unresolved mappings to fall back silently to similarly named fields.
- Removing historical generated documents during correction or regeneration.

19. Lessons Learned

19.1 Schema migrations are contract migrations

When a database represents behavior consumed by templates and resolvers, changing the schema changes contracts. The migration plan must include every consumer, not only the source and target tables.

19.2 Domain ownership accelerates debugging

The fastest path to root cause combined technical evidence with knowledge of Affordable Housing document behavior. Deep module ownership made it possible to distinguish a data problem from a mapping problem, resolver problem, or common-platform defect.

19.3 Customer-visible output needs end-to-end validation

Unit and database tests are important, but the final rendered document is the result the customer sees. Representative generation and comparison should be a release gate for document-platform changes.

19.4 Cross-team dependencies must be discoverable

A shared schema creates organizational dependencies. Release notes, mapping ownership, compatibility matrices, and escalation paths are engineering controls, not administrative overhead.

19.5 Every production defect should become a regression case

The six-bug day demonstrated that urgent work can either remain repeated operational cost or be converted into durable coverage. Each confirmed root cause should strengthen fixtures, validations, or observability.

19.6 Long-lived systems need explicit deprecation

Legacy fields and templates persist when there is no verified retirement process. A migration is complete only when compatibility paths can be removed safely based on evidence.

20. Modernization Direction

Status boundary
The following items describe how the platform could be modernized today. They are not claims about the historical implementation.

20.1 Versioned field registry

A centralized registry could assign stable canonical identifiers, owners, schemas, supported contexts, lifecycle status, and version history to every merge field. Templates would reference the registry contract rather than physical database structure.

20.2 Typed resolver interfaces

Resolvers could implement typed contracts that declare inputs, output type, null semantics, formatting responsibility, and authorization requirements. Contract tests would verify that resolver behavior remains compatible across releases.

20.3 Automated document diffing

A regression service could generate baseline and candidate documents from synthetic fixtures, extract resolved values, render both files, and produce visual and semantic diffs. Approved expected differences would be versioned with the test.

20.4 Event-driven generation for heavy workflows

Long-running or bulk document operations could be moved to asynchronous workers with explicit job state, retry policy, idempotent generation keys, and operational dashboards. Interactive single-document generation could remain synchronous when latency is acceptable.

20.5 Stronger observability

OpenTelemetry-style tracing could connect the template request, field-resolution spans, database calls, rendering, storage, and delivery. A support engineer could inspect one trace instead of correlating several incomplete logs.

20.6 Safe AI assistance

AI could assist support teams by retrieving template documentation, similar incidents, mapping history, and resolver ownership. It should not invent field values or modify production mappings automatically. Deterministic definitions and authorized business data remain the source of truth.

20.7 Continuous migration readiness

Rather than treating migration as a one-time event, the platform could continuously report deprecated fields, unsupported template references, unowned definitions, resolver-contract drift, and templates without regression fixtures.

Appendix A. Migration Checklist

☐ Inventory active templates and every referenced merge field.

☐ Assign each legacy field a canonical target, retirement decision, or documented exception.

☐ Validate identifiers, aliases, data types, formatting, and resolver ownership.

☐ Confirm application/schema compatibility and deployment order.

☐ Build representative fixtures for high-value and high-risk document scenarios.

☐ Generate and compare output before cutover.

☐ Prepare smoke tests, rollback scripts, and diagnostic queries.

☐ Monitor unresolved fields, generation failures, and legacy-path usage.

☐ Reconcile production defects to the migration inventory.

☐ Preserve audit history when documents are regenerated.

☐ Remove compatibility behavior only after usage reaches a verified safe state.

Appendix B. Glossary

Term	Definition
Document template	A reusable layout containing static content and merge-field references.
Merge field	A placeholder contract resolved to a context-specific value during generation.
Field catalog	The registry or schema containing merge-field definitions and metadata.
Resolver	Application or database logic that obtains or derives a field value.
Canonical field	The approved target definition representing one stable business meaning.
Compatibility layer	Temporary logic allowing legacy references to resolve through the new model.
Semantic equivalence	Preservation of business meaning, not merely structural similarity.
Rendered-output validation	Testing the generated document, including visible values and formatting.
Regeneration	Producing corrected output after the original document was generated.
Blast radius	Templates, documents, customers, or workflows potentially affected by a defect.